What the *i2c_register_board_info* does is link the *struct i2c_board_info* object in *__i2c_board_list*, which is the global linked list.

Now, when the I2C adapter is registered using *i2c_register_adapter* API (defined in drivers/i2c/i2c-core.c), it will search for devices that have the same bus number as the adapter, through the *i2c_scan_static_board_info* API. When the *i2c_board_info* object is found with the bus number which is the same as that of the adapter being registered, a new *i2c_client* object is created using the *i2c_new_device* API.

The *i2c_new_device* API creates a new *struct i2c_client* object and the fields of the *i2c_client* object are initialised with the fields of the *i2c_board_info* object. The new *i2c_client* object is then registered with the I2C subsystem. During registration, the kernel matches the name of all the I2C drivers with the name of the I2C client created. If any I2C driver's name matches with the I2C client, then the probe routine of the I2C driver will be called.

Case 2: In some cases, instead of the bus number, the *i2c_adapter* on which the device is connected is known; in this case, the device is registered using the *struct i2c_adapter* object.

When the *i2c_adapter* is known instead of the bus number, the I2C device is registered using the following API:

```
struct i2c_client *
 i2c_new_device(struct i2c_adapter *adap, struct i2c_board_
info const *info);
```

…where,

adap = *i2c_adapter* representing the bus on which the device is connected.

info = *i2c_board_info* object for each device.

In our example, the device is registered as follows.

For device 1:

```
i2c_new_device(adap, &xyz_devices[0]);
```

For device 2:

```
i2c_new_device(adap, &xyz_devices[1]);
```

Writing the I2C driver

As mentioned earlier, generally, the device files are present in the *arch/xyz_arch/boards* folder and similarly, the driver files reside in their respective driver folders. For example, typically, all the RTC drivers reside in the *drivers/rtc* folder and all the keyboard drivers reside in the *drivers/input/keyboard* folder.

Writing the I2C driver involves specifying the details of the *struct i2c_driver*. The following are the required fields for *struct i2c_driver* that need to be filled:

driver.name = name of the driver that will be used to match the driver with the device.

driver.owner = owner of the module. This is generally the *THIS_MODULE* macro.

probe = the probe routine for the driver, which will be called when any I2C device's name in the system matches with this driver's name.

Note: It's not just the names of the device and driver that are used to match the two. There are other methods to match them such as *id_table* but, for now, let's consider their names as the main parameter for matching. To understand the way in which the ID table is used, refer to the Linux source code.

For our example of the EEPROM driver, the driver file will reside in the *drivers/misc/eeprom* folder and we will give it a name—*eeprom_xyz.c*. The *struct i2c_driver* will be written as follows:

```
static struct i2c_driver eeprom_driver = {
            .driver = {
                        .name = "eeprom_xyz",
       .owner = THIS_MODULE,
     },
     .probe = eeprom_probe,
};
```

For our example of an *adc* driver, the driver file will reside in the *drivers/iio/adc* folder, which we will name as *adc_xyz.c*, and the *struct i2c_driver* will be written as follows:

```
static struct i2c_driver adc_driver = {
      .driver = {
        .name = "adc_xyz",
        .owner = THIS_MODULE,
      },
      .probe = adc_probe,
};
```

The *struct i2c_driver* now has to be registered with the I2C subsystem. This is done in the *module_init* routine using the following API:

```
i2c_add_driver(struct i2c_driver *drv);
```

…where *drv* is the *i2c_driver* structure written for the device.

For our example of an EEPROM system, the driver will be registered as:

```
i2c_add_driver(&eeprom_driver);
```

…and the *adc* driver will be registered as:

```
i2c_add_driver(&adc_driver);
```

What this *i2c_add_driver* does is register the passed driver with the I2C subsystem and match the name of the driver with all